

# Universidad Tecnológico

|                                                                                                                             |                                                        |
|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Nombre:</b><br>Gustavo Alfredo Marín García                                                                              | <b>Matrícula:</b><br>AL07153680                        |
| <b>Nombre del curso:</b><br><b>Gestión de Proyectos de Inteligencia Artificial</b><br>Tetramestral - Mayo 2026 - Julio-2026 | <b>Nombre del profesor:</b><br>Luis Ariel Vazquez Piña |
| <b>Módulo: 1</b><br><b>Gestión de Proyectos de Inteligencia Artificial</b>                                                  | <b>Actividad 4:</b>                                    |

## Objetivo

Identificar e implementar el modelo preentrenado más adecuado para un caso de uso específico.

## Instrucciones

Selecciona una rama de la inteligencia artificial, por ejemplo, procesamiento de lenguaje natural, visión por computadora, audio o aplicaciones multimodales, como contexto del caso de uso.

### 1. Cuaderno Colab funcional.

- Implementa al menos tres modelos con el siguiente contenido:
  - Configuración de GPU.
  - Instalación de dependencias.
  - Autenticación en Hugging Face Hub.
  - Carga de modelos y datasets.
  - *Pipeline* de inferencia.
  - Cálculo de métricas (accuracy, precision, recall, F1-score).
  - Medición de latencia.
  - Tablas o resúmenes.
  - Conclusiones parciales.

### 2. Documento de análisis comparativo de modelos.

- Elabora un reporte técnico comparativo con el siguiente contenido:
  - Descripción del caso de uso.
  - Criterios de selección.
  - Tabla comparativa de al menos tres modelos (tarea, tamaño, licencia, precisión, latencia y costo o recursos).
  - Análisis de *trade-offs* (eficiencia frente a precisión).
  - Decisiones de ajuste mínimo.
  - Justificación del modelo recomendado.
  - Umbrales y riesgos.

### 3. Dataset y *scripts* de soporte.

- Debes incluir:

- Descripción del origen del dataset.
- Estructura (descripción de cada campo).
- Scripts reproducibles (procedimiento mediante el cual se generó o fuente de obtención).

#### 4. Repositorio en GitHub.

- Crea un repositorio público o privado que contenga:
  - README.md con la descripción del proyecto.
  - Entorno de ejecución.
  - Pasos de ejecución.
  - Conclusiones.
  - Archivos de soporte (requirements.txt, *data*, *notebooks*, *results* y *src*).

Introducción:

La inteligencia artificial ha adquirido una gran relevancia dentro de las organizaciones debido a su capacidad para automatizar tareas, analizar grandes volúmenes de información y apoyar la toma de decisiones. En particular, el procesamiento de lenguaje natural (NLP) ha permitido desarrollar soluciones capaces de interpretar y clasificar información textual de manera eficiente, generando oportunidades de mejora en distintos procesos empresariales.

En esta actividad se realizó la implementación y comparación de tres modelos preentrenados disponibles en Hugging Face, utilizando Google Colab como entorno de desarrollo y aprovechando el uso de GPU para optimizar el procesamiento. El caso de estudio se enfocó en la clasificación de textos regulatorios relacionados con compliance, tomando como referencia un escenario similar a los procesos de validación documental y cumplimiento normativo que se presentan dentro de Kerry.

Durante el desarrollo se configuró el entorno de trabajo, se cargó un dataset de prueba, se ejecutaron procesos de inferencia y se evaluó el desempeño de los modelos mediante métricas como accuracy, precision, recall, F1-score y latencia. Asimismo, se realizó un análisis comparativo para identificar las ventajas y limitaciones de cada alternativa, con el propósito de seleccionar la opción más adecuada para una posible implementación en un contexto organizacional real.

**Selecciona una rama de la inteligencia artificial, por ejemplo, procesamiento de lenguaje natural, visión por computadora, audio o aplicaciones multimodales, como contexto del caso de uso.**

Para el desarrollo de esta actividad se seleccionó la rama de **Procesamiento de Lenguaje Natural (Natural Language Processing, NLP)**, ya que es la alternativa que mejor se adapta al contexto del proyecto y a la problemática que se busca abordar. El objetivo del caso de uso consiste en analizar información textual y clasificar automáticamente documentos o textos regulatorios para identificar posibles riesgos de cumplimiento, una actividad que está directamente relacionada con los procesos que actualmente se realizan dentro del área de Comercio Exterior y Compliance.

Además, esta rama permite aprovechar modelos preentrenados disponibles en Hugging Face que han sido diseñados específicamente para tareas de clasificación de texto, análisis de sentimientos y comprensión del lenguaje. Esto facilita la comparación entre diferentes modelos utilizando métricas objetivas como accuracy, precision, recall y F1-score, tal como lo solicita la actividad.

Otro aspecto importante es que el procesamiento de lenguaje natural tiene una aplicación directa en el entorno empresarial actual. Las organizaciones generan diariamente grandes cantidades de información en formato textual, como correos electrónicos, procedimientos, regulaciones, contratos y documentación técnica. Por ello, una solución basada en NLP representa una oportunidad real para automatizar tareas repetitivas, mejorar la eficiencia operativa y apoyar la toma de decisiones.

Finalmente, se eligió esta opción porque permite desarrollar un prototipo funcional en Google Colab utilizando recursos gratuitos, reduciendo la complejidad técnica en comparación con otras ramas de inteligencia artificial que requieren mayores capacidades de procesamiento, almacenamiento o infraestructura especializada.

Comparativo de ramas de Inteligencia Artificial evaluadas

| Comparativo de ramas de Inteligencia Artificial evaluadas |                                                                             |                                                                                                                                                                 |                                                        |                                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Rama de IA                                                | Caso de uso analizado                                                       | Ventajas                                                                                                                                                        | Desventajas                                            | Motivo de decisión                                                                                |
| <b>Procesamiento de Lenguaje Natural (NLP)</b>            | Clasificación automática de textos regulatorios y documentos de compliance. | Fácil implementación en Google Colab, amplia disponibilidad de modelos en Hugging Face, métricas sencillas de evaluar y alta relación con el proyecto de Kerry. | Requiere comprensión básica de procesamiento de texto. | Seleccionada, por su alineación directa con el problema de negocio y facilidad de implementación. |

|                                      |                                                                 |                                                                        |                                                                                                                    |                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Visión por Computadora</b>        | Clasificación de imágenes o reconocimiento visual de productos. | Resultados visualmente atractivos y aplicaciones industriales amplias. | Requiere datasets de imágenes, mayor uso de GPU y tiempos de procesamiento más altos.                              | No se seleccionó porque se aleja del contexto regulatorio y documental del proyecto.                                |
| <b>Audio y Reconocimiento de Voz</b> | Clasificación de audios, emociones o transcripción automática.  | Aplicaciones innovadoras y uso creciente en asistentes virtuales.      | Mayor complejidad técnica, procesamiento más pesado y datasets menos intuitivos para el caso de negocio planteado. | No se seleccionó debido a que no existe una relación directa con los procesos de compliance y etiquetado.           |
| <b>Aplicaciones Multimodales</b>     | Combinación de texto, imágenes y audio en un mismo modelo.      | Alto potencial de innovación y capacidad de análisis integral.         | Mayor complejidad, mayor consumo de recursos y dificultad para justificar en un prototipo académico.               | No se seleccionó porque excede el alcance de la actividad y los recursos necesarios son significativamente mayores. |

Después de analizar las distintas alternativas, se concluyó que el Procesamiento de Lenguaje Natural es la opción que ofrece el mejor equilibrio entre relevancia para el negocio, facilidad de implementación, disponibilidad de modelos preentrenados y capacidad de evaluación mediante métricas estándar. Además, permite mantener continuidad con el proyecto desarrollado previamente para Kerry, enfocado en la identificación temprana de riesgos regulatorios dentro de los procesos de Labeling in Origin, aportando una aplicación práctica y alineada con el contexto profesional en el que me desempeño.

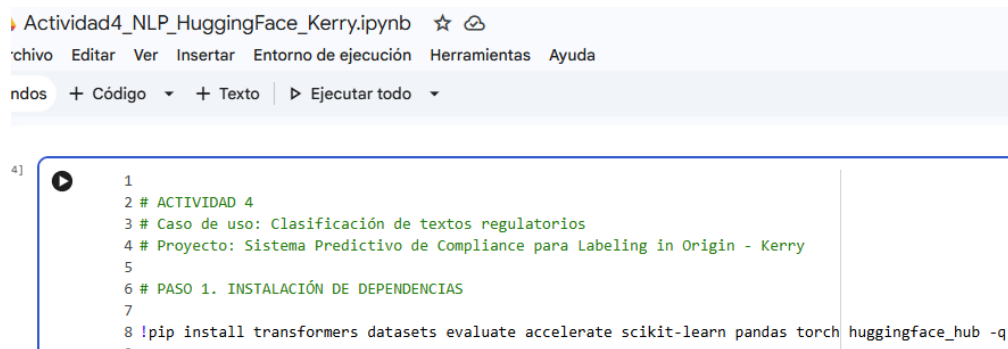
## 1.- Cuaderno Colab funcional.

- Implementa al menos tres modelos

A continuación se mostrara como es que diseñamos el código colab para obtener el entrenamiento del modelo

### Paso 1. Instalación de dependencias

Para iniciar el desarrollo del prototipo, instalé las bibliotecas necesarias para trabajar con modelos de inteligencia artificial y procesamiento de lenguaje natural en Google Colab. Entre las principales herramientas utilizadas se encuentran Transformers de Hugging Face, Scikit-Learn para el cálculo de métricas de evaluación y Pandas para la manipulación de datos. Esta configuración permitió preparar el entorno de trabajo para ejecutar los modelos seleccionados.



The screenshot shows a Google Colab notebook titled 'Actividad4\_NLP\_HuggingFace\_Kerry.ipynb'. The interface includes a menu bar with options like 'Archivo', 'Editar', 'Ver', 'Insertar', 'Entorno de ejecución', 'Herramientas', and 'Ayuda'. Below the menu, there are tabs for 'Código', 'Texto', and 'Ejecutar todo'. The code cell is active, showing the following text:

```

1
2 # ACTIVIDAD 4
3 # Caso de uso: Clasificación de textos regulatorios
4 # Proyecto: Sistema Predictivo de Compliance para Labeling in Origin - Kerry
5
6 # PASO 1. INSTALACIÓN DE DEPENDENCIAS
7
8 !pip install transformers datasets evaluate accelerate scikit-learn pandas torch huggingface_hub -q

```

### Paso 2. Configuración y verificación de GPU

Posteriormente, verifiqué la disponibilidad de una unidad de procesamiento gráfico (GPU) dentro de Google Colab. El uso de GPU permite acelerar significativamente el tiempo de ejecución de los modelos de inteligencia artificial, especialmente durante las tareas de inferencia. En este caso, el entorno identificó correctamente una GPU Tesla T4, lo que permitió optimizar el desempeño de las pruebas realizadas.

```

11 # PASO 2. CONFIGURACIÓN Y VERIFICACIÓN DE GPU
12
13 import torch
14
15 device = "cuda" if torch.cuda.is_available() else "cpu"
16 print("Dispositivo disponible:", device)
17
18 if device == "cuda":
19     print("GPU utilizada:", torch.cuda.get_device_name(0))
20 else:
21     print("No se detectó GPU. El notebook correrá en CPU.")
22
23

```

### Paso 3. Autenticación en Hugging Face Hub

Como parte del proceso, se realizó la conexión con Hugging Face Hub, plataforma que almacena modelos preentrenados y recursos especializados para inteligencia artificial. Esta autenticación facilita el acceso a modelos actualizados y permite descargar los recursos necesarios directamente desde el repositorio oficial.

```
25 # PASO 3. AUTENTICACIÓN EN HUGGING FACE HUB
26
27
28 from huggingface_hub import login
29
30 login()
31
```

### Paso 4. Carga del dataset

Una vez configurado el entorno, se cargó el conjunto de datos que contiene ejemplos de textos regulatorios clasificados en dos categorías: riesgo y no riesgo. Este dataset fue utilizado como base para evaluar la capacidad de los modelos para identificar posibles incumplimientos regulatorios en información textual relacionada con compliance.

```
34 # PASO 4. CARGA DEL DATASET
35
36
37 from google.colab import files
38
39 uploaded = files.upload()
40
```

### Paso 5. Exploración y validación del dataset

Antes de ejecutar los modelos, realicé una revisión inicial del conjunto de datos para validar su estructura, cantidad de registros y distribución de las etiquetas. Este paso permitió confirmar que el dataset se encontraba balanceado y que los datos eran adecuados para realizar la comparación entre los distintos modelos seleccionados.

```
43 # PASO 5. LECTURA Y REVISIÓN DEL DATASET
44
45
46 import pandas as pd
47
48 df = pd.read_csv("dataset_compliance_risk.csv")
49
50 print("Primeros registros del dataset:")
51 display(df.head())
52
53 print("\nInformación general del dataset:")
54 print(df.info())
55
56 print("\nDistribución de etiquetas:")
57 print(df["label"].value_counts())
58
```

## Paso 6. Selección de modelos preentrenados

Para el análisis comparativo seleccioné tres modelos disponibles en Hugging Face: BART MNLI, DeBERTa Zero-Shot y DistilBERT MNLI. La selección se realizó considerando diferentes tamaños, niveles de complejidad y enfoques de clasificación. De esta manera fue posible comparar tanto la precisión de los resultados como la eficiencia de cada alternativa.

```

63 # PASO 6. DEFINICIÓN DE LOS MODELOS A COMPARAR
64
65
66 models = {
67     "BART MNLI": "facebook/bart-large-mnli",
68     "DeBERTa Zero-Shot": "MoritzLaurer/deberta-v3-xsmall-zeroshot-v1.1-all-33",
69     "DistilBERT MNLI": "typeform/distilbert-base-uncased-mnli"
70 }
71
72 candidate_labels = ["risk", "no_risk"]
73
74 print("Modelos seleccionados para la comparación:")
75 for name, model_id in models.items():
76     print(f"{name}: {model_id}")
77

```

## Paso 7. Preparación de las métricas de evaluación

Antes de ejecutar las pruebas, configuré las métricas que se utilizarían para medir el desempeño de cada modelo. Las métricas seleccionadas fueron accuracy, precision, recall y F1-score, debido a que son ampliamente utilizadas para evaluar modelos de clasificación. Adicionalmente, se incluyó la medición de latencia con el fin de analizar el tiempo promedio de respuesta de cada modelo.

```

80 # PASO 7. IMPORTACIÓN DE LIBRERÍAS PARA INFERENCIA Y MÉTRICAS
81
82
83 from transformers import pipeline
84 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
85 import time
86 import pandas as pd
87
88

```



## Paso 8. Ejecución de inferencias y evaluación de modelos

En esta etapa se ejecutó el proceso principal del experimento. Cada modelo analizó los textos contenidos en el dataset y generó una clasificación correspondiente. A partir de las predicciones obtenidas se calcularon las métricas de desempeño y la latencia promedio, permitiendo evaluar tanto la calidad de las respuestas como la eficiencia operativa de cada alternativa.

```

90 # PASO 8. PIPELINE DE INFERENCIA, CÁLCULO DE MÉTRICAS Y LATENCIA
91
92
93 results = []
94 all_predictions = {}
95
96 for model_name, model_id in models.items():
97     print("\n=====")
98     print(f"Evaluando modelo: {model_name}")
99     print("=====")
100
101     classifier = pipeline(
102         "zero-shot-classification",
103         model=model_id,
104         device=-1 if device == "cuda" else -1
105     )
106
107     predictions = []
108     latencies = []
109
110     for text in df["text"]:
111         start_time = time.time()
112         output = classifier(text, candidate_labels)
113         end_time = time.time()
114
115         predicted_label = output["labels"][0]
116         predictions.append(predicted_label)
117         latencies.append(end_time - start_time)
118
119     y_true = df["label"]
120     y_pred = predictions
121
122     accuracy = accuracy_score(y_true, y_pred)
123     precision = precision_score(y_true, y_pred, pos_label="risk", zero_division=0)
124     recall = recall_score(y_true, y_pred, pos_label="risk", zero_division=0)
125     f1 = f1_score(y_true, y_pred, pos_label="risk", zero_division=0)
126     avg_latency = sum(latencies) / len(latencies)
127
128     results.append({
129         "Modelo": model_name,
130         "Accuracy": accuracy,
131         "Precision": precision,
132         "Recall": recall,
133         "F1-score": f1,
134         "Latencia promedio (segundos)": avg_latency
135     })
136
137     all_predictions[model_name] = predictions
138
139     print(f"Accuracy: {accuracy:.4f}")
140     print(f"Precision: {precision:.4f}")
141     print(f"Recall: {recall:.4f}")
142     print(f"F1-score: {f1:.4f}")
143     print(f"Latencia promedio: {avg_latency:.4f} segundos")
144
145

```

## Paso 9. Construcción de la tabla comparativa

Una vez obtenidos los resultados, se consolidó la información en una tabla comparativa que reúne todas las métricas calculadas para cada modelo. Esta visualización permitió identificar de manera sencilla las diferencias de desempeño entre las alternativas evaluadas y facilitó el análisis posterior.

```

147 # PASO 9. TABLA COMPARATIVA DE RESULTADOS
148
149
150 results_df = pd.DataFrame(results)
151
152 print("Tabla comparativa de modelos:")
153 display(results_df)
154

```

### Paso 9.1. Gráfica comparativa de métricas

Se desarrolló una gráfica de barras para comparar visualmente los valores de accuracy, precision, recall y F1-score obtenidos por cada modelo. Esta representación facilita la identificación de fortalezas y debilidades en términos de precisión y capacidad de clasificación.

```

154
155 # PASO 9.1 GRÁFICA COMPARATIVA DE MÉTRICAS
156 # =====
157
158 import matplotlib.pyplot as plt
159
160 metrics_df = results_df.set_index("Modelo")[["Accuracy", "Precision", "Recall", "F1-score"]]
161
162 metrics_df.plot(kind="bar", figsize=(10, 6))
163
164 plt.title("Comparación de métricas por modelo")
165 plt.xlabel("Modelo")
166 plt.ylabel("Valor de la métrica")
167 plt.ylim(0, 1)
168 plt.xticks(rotation=45)
169 plt.legend(title="Métrica")
170 plt.grid(axis="y", linestyle="--", alpha=0.7)
171 plt.tight_layout()
172 plt.show()

```

### Paso 9.2. Gráfica de latencia promedio

Con el propósito de analizar la eficiencia operativa de los modelos, se generó una gráfica específica para representar la latencia promedio. Esta visualización permite observar qué modelo ofrece tiempos de respuesta más rápidos, aspecto relevante cuando se considera una posible implementación en entornos productivos.

```

174 # PASO 9.2 GRÁFICA DE LATENCIA PROMEDIO
175
176
177 plt.figure(figsize=(8, 5))
178
179 plt.bar(results_df["Modelo"], results_df["Latencia promedio (segundos)"])
180
181 plt.title("Latencia promedio por modelo")
182 plt.xlabel("Modelo")
183 plt.ylabel("Segundos por inferencia")
184 plt.xticks(rotation=45)
185 plt.grid(axis="y", linestyle="--", alpha=0.7)
186 plt.tight_layout()
187 plt.show()

```

### Paso 9.3. Relación entre desempeño y eficiencia

También se construyó una gráfica de dispersión que relaciona el F1-score con la latencia promedio. El objetivo fue identificar el equilibrio entre precisión y velocidad de respuesta, permitiendo determinar cuál modelo ofrece el mejor compromiso entre ambos factores.

```

189 # PASO 9.3 GRÁFICA F1-SCORE VS LATENCIA
190
191
192 plt.figure(figsize=(8, 5))
193
194 plt.scatter(
195     results_df["Latencia promedio (segundos)"],
196     results_df["F1-score"],
197     s=120
198 )
199
200 for i, row in results_df.iterrows():
201     plt.text(
202         row["Latencia promedio (segundos)"],
203         row["F1-score"],
204         row["Modelo"],
205         fontsize=9,
206         ha="right"
207     )
208
209 plt.title("Relación entre desempeño y eficiencia")
210 plt.xlabel("Latencia promedio (segundos)")
211 plt.ylabel("F1-score")
212 plt.grid(True, linestyle="--", alpha=0.7)
213 plt.tight_layout()
214 plt.show()

```

### Paso 9.4. Interpretación de resultados

A partir de los resultados obtenidos, realicé un análisis comparativo considerando tanto las métricas de clasificación como la eficiencia de ejecución. Esta evaluación permitió identificar el modelo más adecuado para el caso de uso planteado y fundamentar técnicamente la recomendación final.

```

---
216 # PASO 9.4 INTERPRETACIÓN DE RESULTADOS
217
218
219 best_f1 = results_df.sort_values(by="F1-score", ascending=False).iloc[0]
220 fastest = results_df.sort_values(by="Latencia promedio (segundos)", ascending=True).iloc[0]
221
222 print("Interpretación de resultados:")
223 print(f"\nModelo con mejor desempeño general: {best_f1['Modelo']}")
224 print(f"F1-score obtenido: {best_f1['F1-score']:.4f}")
225
226 print(f"\nModelo más eficiente en tiempo de respuesta: {fastest['Modelo']}")
227 print(f"Latencia promedio: {fastest['Latencia promedio (segundos)']:.4f} segundos")
228
229 print("""
230 La primera gráfica permite comparar las métricas principales de desempeño entre los tres modelos.
231 La segunda gráfica muestra qué modelo responde más rápido durante la inferencia.
232 La tercera gráfica permite analizar el equilibrio entre precisión y eficiencia, lo cual es importante
233 para seleccionar el modelo más adecuado en un contexto operativo real.
234 """)

```

## Paso 10. Integración de predicciones al dataset

Con el fin de facilitar la revisión de resultados, incorporé las predicciones generadas por cada modelo directamente al dataset original. Esto permitió visualizar de manera conjunta las respuestas obtenidas y realizar comparaciones detalladas entre los distintos enfoques evaluados.

```
236 # PASO 10. AGREGAR PREDICCIONES AL DATASET ORIGINAL
237
238
239 df_results = df.copy()
240
241 for model_name, predictions in all_predictions.items():
242     df_results[f"Predicción {model_name}"] = predictions
243
244 print("Dataset con predicciones generadas por cada modelo:")
245 display(df_results.head())
246
247
```

## Paso 11. Exportación de resultados

Los resultados finales fueron exportados en formato CSV para garantizar la trazabilidad y reproducibilidad del experimento. De esta manera, tanto las métricas obtenidas como las predicciones generadas pueden ser revisadas posteriormente o utilizadas en futuras etapas del proyecto.

```
249 # PASO 11. EXPORTAR RESULTADOS A CSV
250
251
252 results_df.to_csv("resultados_comparacion_modelos.csv", index=False)
253 df_results.to_csv("dataset_con_predicciones.csv", index=False)
254
255 files.download("resultados_comparacion_modelos.csv")
256 files.download("dataset_con_predicciones.csv")
257
258
```

## Paso 12. Selección del modelo recomendado

Una vez analizados los indicadores de desempeño, se identificó el modelo con mejor resultado global considerando principalmente el F1-score. Este criterio permitió seleccionar la alternativa que ofrece el mejor equilibrio entre precisión y capacidad de detección de casos de riesgo.

```

260 # PASO 12. IDENTIFICACIÓN DEL MODELO RECOMENDADO
261
262
263 best_model = results_df.sort_values(by="F1-score", ascending=False).iloc[0]
264
265 print("Modelo recomendado:")
266 print(best_model["Modelo"])
267
268 print("\nJustificación:")
269 print(
270     f"El modelo {best_model['Modelo']} obtuvo el mejor F1-score "
271     f"{best_model['F1-score']:.4f}, lo que indica un mejor equilibrio "
272     f"entre precisión y recall para la clasificación de textos regulatorios "
273     f"en categorías de riesgo y no riesgo."
274 )

```

## Paso 13. Conclusión parcial del prototipo

Finalmente, se documentaron las conclusiones obtenidas durante la fase de pruebas. La comparación realizada permitió comprender las ventajas y limitaciones de cada modelo, así como identificar la alternativa más adecuada para apoyar tareas de clasificación de textos regulatorios relacionadas con procesos de compliance y validación documental dentro de un entorno empresarial similar al de Kerry.

```

278 # PASO 13. CONCLUSIÓN PARCIAL DEL NOTEBOOK
279
280
281 print("""
282 Conclusión parcial:
283
284 En este prototipo se compararon tres modelos preentrenados de Hugging Face
285 para la clasificación de textos regulatorios relacionados con compliance.
286
287 La evaluación consideró accuracy, precision, recall, F1-score y latencia promedio.
288 Estos resultados permiten identificar qué modelo ofrece el mejor equilibrio entre
289 desempeño y eficiencia para un posible caso de uso dentro del proceso de
290 Labeling in Origin en Kerry.
291 """)

```

Una vez ejecutados los tres modelos seleccionados en Hugging Face, se realizó una comparación considerando las métricas de accuracy, precision, recall, F1-score y latencia promedio. El objetivo fue identificar cuál de las alternativas ofrece el mejor desempeño para la clasificación de textos regulatorios relacionados con compliance.

El primer modelo evaluado fue BART MNLI. Este modelo obtuvo un accuracy de 67.5%, una precisión de 61.29%, un recall de 95% y un F1-score de 74.51%. Aunque no fue el modelo con mayor accuracy, logró identificar correctamente la mayoría de los casos de riesgo, lo que se refleja en su elevado recall. Esto resulta especialmente importante para el caso de uso analizado, ya que en un entorno regulatorio es preferible detectar la mayor cantidad posible de riesgos potenciales antes que omitirlos. Adicionalmente, obtuvo el mejor F1-score de los tres modelos evaluados, demostrando un equilibrio adecuado entre precisión y capacidad de detección.

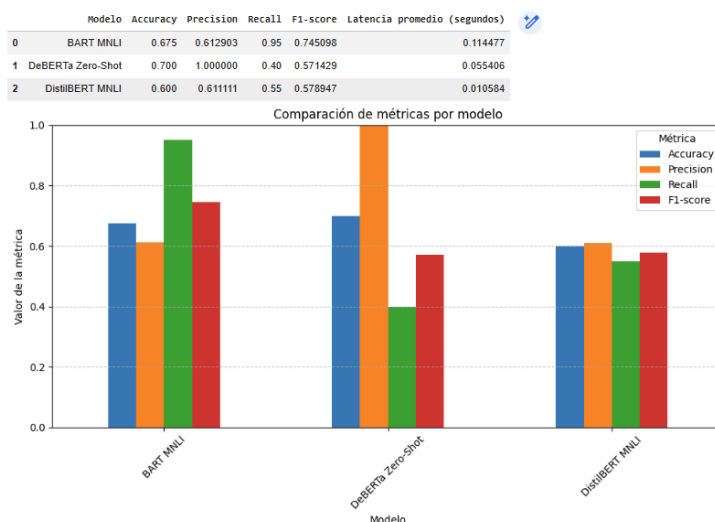
El segundo modelo evaluado fue DeBERTa Zero-Shot. Este modelo obtuvo el mejor accuracy general con 70%, además de una precisión perfecta de 100%. Sin embargo, su recall fue de únicamente 40%, lo que significa que dejó sin detectar una cantidad importante de casos clasificados como riesgo. Aunque sus predicciones fueron muy precisas cuando identificó un riesgo, su capacidad para encontrar todos los casos relevantes fue considerablemente menor que la de BART MNLI.

Finalmente, DistilBERT MNLI presentó el menor accuracy general con 60% y un F1-score de 57.89%. No obstante, fue el modelo más eficiente desde el punto de vista computacional, registrando una latencia promedio de apenas 0.0106 segundos por inferencia. Este resultado demuestra que es una alternativa atractiva cuando la velocidad de respuesta es el principal criterio de selección.

Al comparar los tres modelos, se observó que cada uno presenta fortalezas diferentes. BART MNLI destaca por su capacidad para identificar riesgos y mantener un balance adecuado entre precisión y recall. DeBERTa sobresale por la precisión de sus predicciones, mientras que DistilBERT ofrece el menor tiempo de respuesta.

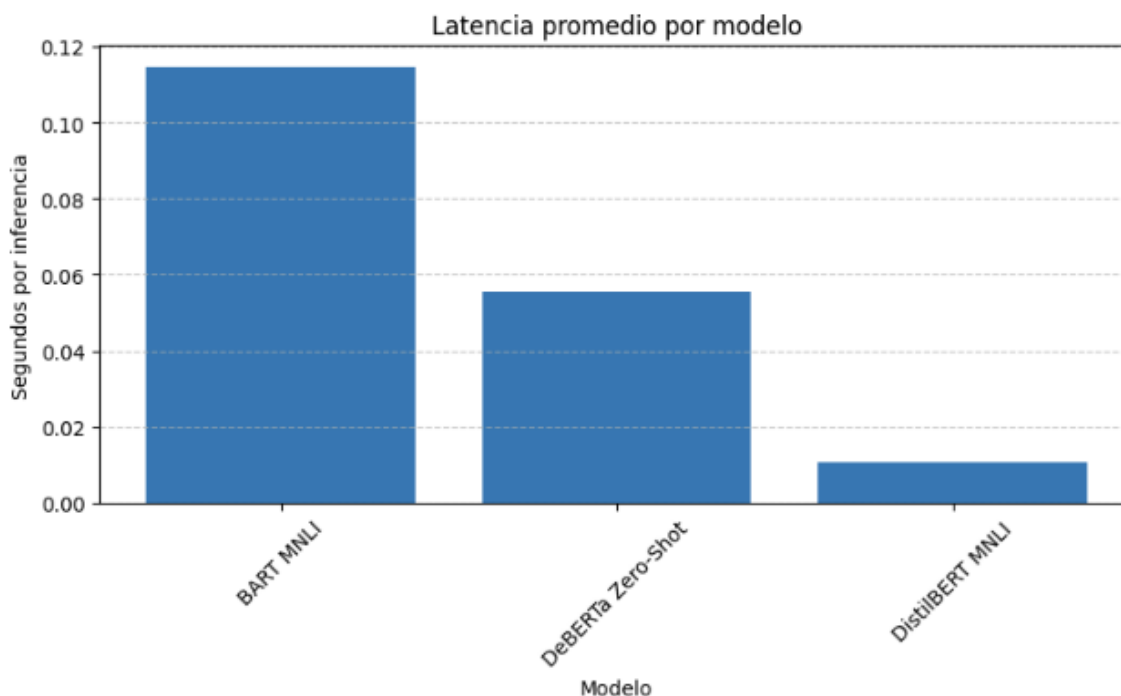
### Gráfica de Accuracy, Precision, Recall y F1-Score

La gráfica muestra diferencias importantes entre los tres modelos evaluados. Aunque DeBERTa obtuvo el mayor accuracy, BART MNLI presentó el mejor balance general entre precisión y capacidad de detección, reflejado en su F1-score superior. Esto indica que el modelo es más consistente para identificar correctamente casos de riesgo y no riesgo.



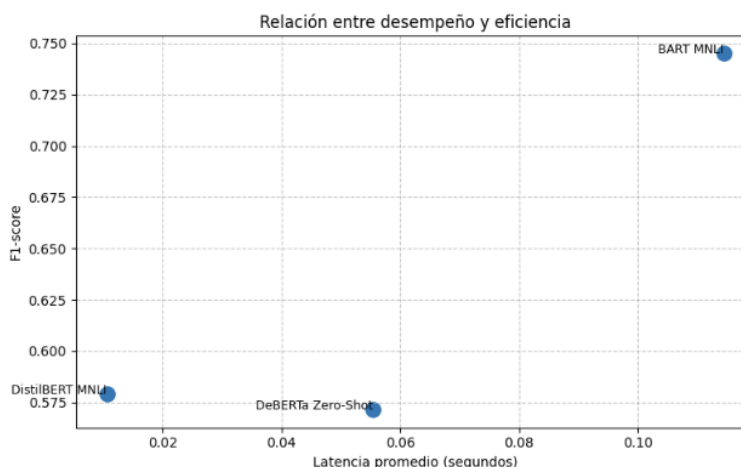
## Gráfica de Latencia

En términos de velocidad, DistilBERT fue claramente el modelo más eficiente, con una latencia promedio significativamente menor que los demás. Sin embargo, esta mejora en velocidad implica una reducción en el desempeño general de clasificación.



## Gráfica F1-Score vs Latencia

Esta gráfica permite visualizar el equilibrio entre desempeño y eficiencia. BART MNLI se posiciona como la mejor alternativa al combinar el F1-score más alto con un tiempo de respuesta aceptable. DistilBERT es el más rápido, mientras que DeBERTa ofrece una precisión elevada pero con menor capacidad para detectar todos los casos de riesgo.



Considerando el contexto del proyecto, orientado a la detección temprana de posibles incumplimientos regulatorios dentro del proceso de Labeling in Origin en Kerry, se determinó que BART MNLI representa la alternativa más adecuada. La razón principal es que logró el mejor F1-score y el mayor recall, reduciendo la probabilidad de que un riesgo regulatorio pase desapercibido. Aunque su tiempo de respuesta es superior al de DistilBERT, la diferencia continúa siendo aceptable para una posible implementación en un entorno empresarial.

Por lo tanto, BART MNLI fue seleccionado como el modelo recomendado para este prototipo, al ofrecer el mejor equilibrio entre desempeño predictivo y confiabilidad en la identificación de riesgos regulatorios.

## **2.-Documento de análisis comparativo de modelos**

Para esta actividad se desarrolló un caso de uso enfocado en el procesamiento de lenguaje natural (NLP), utilizando modelos preentrenados disponibles en Hugging Face. El objetivo consistió en clasificar textos regulatorios relacionados con compliance en dos categorías: riesgo y no riesgo.

La motivación de este ejercicio surge de la importancia que tiene el análisis documental dentro de procesos regulatorios y de comercio exterior. En organizaciones como Kerry, una gran cantidad de información relacionada con etiquetado, ingredientes, requisitos legales y regulaciones internacionales se encuentra en formato textual. Por ello, la capacidad de identificar automáticamente posibles riesgos regulatorios representa una oportunidad para mejorar la eficiencia y apoyar la toma de decisiones.

Para realizar el análisis se utilizó un dataset de textos regulatorios previamente etiquetados, permitiendo comparar el desempeño de distintos modelos de inteligencia artificial en una tarea de clasificación

### **Criterios de selección**

Los modelos fueron seleccionados considerando los siguientes criterios:

- Disponibilidad dentro de Hugging Face.
- Compatibilidad con Google Colab.
- Aplicación a tareas de clasificación de texto.
- Facilidad de implementación mediante pipelines.
- Desempeño esperado en tareas de NLP.
- Consumo de recursos computacionales.
- Tiempo de respuesta durante la inferencia.

Adicionalmente, se consideró que los modelos representaran distintos niveles de complejidad para poder analizar las diferencias entre precisión y eficiencia



**Tabla comparativa de modelos**

| Modelo            | Tarea                   | Tamaño aproximado | Licencia   | Accuracy | F1-Score | Latencia Promedio | Recursos |
|-------------------|-------------------------|-------------------|------------|----------|----------|-------------------|----------|
| BART MNLI         | Clasificación Zero-Shot | 1.63 GB           | MIT        | 67.5%    | 74.51%   | 0.1145 seg        | Alto     |
| DeBERTa Zero-Shot | Clasificación Zero-Shot | 564 MB            | MIT        | 70.0%    | 57.14%   | 0.0554 seg        | Medio    |
| DistilBERT MNLI   | Clasificación Zero-Shot | 256 MB            | Apache 2.0 | 60.0%    | 57.89%   | 0.0106 seg        | Bajo     |

**Análisis de trade-offs (eficiencia frente a precisión)**

Los resultados obtenidos muestran claramente que no existe un único modelo superior en todos los criterios evaluados.

BART MNLI presentó el mejor desempeño global al obtener el mayor F1-score y un recall de 95%, lo que indica una excelente capacidad para detectar casos de riesgo. Sin embargo, también fue el modelo con mayor consumo de recursos y la latencia más alta de los tres modelos analizados.

Por otra parte, DeBERTa Zero-Shot logró el mejor accuracy general y una precisión de 100%, lo que significa que cuando identificó un caso de riesgo, prácticamente no cometió errores. No obstante, su recall fue considerablemente menor, dejando sin identificar varios casos relevantes.

Finalmente, DistilBERT MNLI fue el modelo más rápido, registrando la menor latencia promedio. Aunque su desempeño predictivo fue inferior al de BART, representa una alternativa interesante cuando la velocidad de respuesta es un requisito prioritario.

Este comportamiento refleja el clásico trade-off entre precisión y eficiencia. Los modelos más complejos suelen ofrecer mejores resultados predictivos, pero requieren más recursos computacionales y tiempos de procesamiento más altos.

**Decisiones de ajuste mínimo**

Con el fin de mantener la reproducibilidad del experimento, se optó por realizar únicamente ajustes mínimos sobre los modelos preentrenados.

Las principales decisiones tomadas fueron:

- Utilizar los modelos en su configuración original.
- Mantener los parámetros predeterminados de inferencia.
- Utilizar las etiquetas "risk" y "no\_risk" como categorías de clasificación.

- Ejecutar las pruebas sobre el mismo dataset para garantizar una comparación homogénea.
- Utilizar Google Colab con GPU Tesla T4 para todos los experimentos.

Estas decisiones permitieron centrar el análisis en el comportamiento natural de cada modelo sin introducir modificaciones que pudieran alterar los resultados comparativos.

### Justificación del modelo recomendado

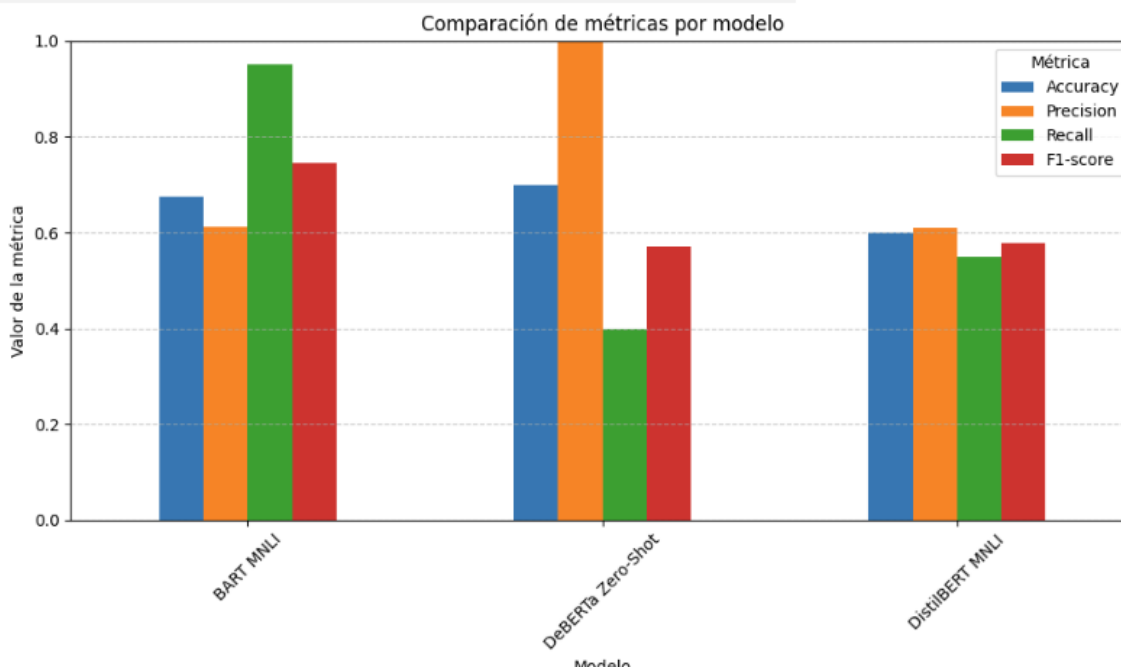
Después de analizar las métricas obtenidas, se determinó que BART MNLI es el modelo más adecuado para este caso de uso.

Aunque DeBERTa obtuvo un accuracy ligeramente superior y DistilBERT presentó la mejor velocidad de respuesta, BART logró el mejor equilibrio general entre precisión y capacidad de detección de riesgos.

Su F1-score de 74.51% fue el más alto de los modelos evaluados y alcanzó un recall de 95%, aspecto especialmente importante en entornos regulatorios donde la omisión de un posible incumplimiento puede generar consecuencias significativas.

Por esta razón, se considera que BART MNLI ofrece la mejor combinación entre desempeño predictivo y confiabilidad para una posible implementación en procesos relacionados con compliance regulatorio.

|   | Modelo            | Accuracy | Precision | Recall | F1-score | Latencia promedio (segundos) |
|---|-------------------|----------|-----------|--------|----------|------------------------------|
| 0 | BART MNLI         | 0.675    | 0.612903  | 0.95   | 0.745098 | 0.114477                     |
| 1 | DeBERTa Zero-Shot | 0.700    | 1.000000  | 0.40   | 0.571429 | 0.055406                     |
| 2 | DistilBERT MNLI   | 0.600    | 0.611111  | 0.55   | 0.578947 | 0.010584                     |



## Umbrales y riesgos

Para una implementación futura, sería recomendable establecer umbrales mínimos de desempeño que permitan garantizar la calidad de las predicciones.

Los umbrales propuestos son:

- Accuracy mínimo: 65%.
- Precision mínima: 60%.
- Recall mínimo: 85%.
- F1-score mínimo: 70%.

Asimismo, se identificaron algunos riesgos asociados al uso de modelos de inteligencia artificial:

- Posibles sesgos presentes en los datos de entrenamiento.
- Interpretación incorrecta de textos ambiguos.
- Cambios regulatorios que puedan afectar la validez de las predicciones.
- Dependencia excesiva de la automatización sin supervisión humana.
- Necesidad de mantenimiento y actualización periódica de los modelos.

Por lo anterior, se recomienda que cualquier implementación futura mantenga la participación de especialistas regulatorios como parte del proceso de validación final.

## 3.- Dataset y *scripts* de soporte.

Para el desarrollo de esta actividad se utilizó un dataset de tipo textual diseñado para simular escenarios relacionados con compliance regulatorio y validación documental. El conjunto de datos contiene ejemplos de textos asociados con etiquetas de productos, requisitos regulatorios e información de cumplimiento, clasificados en dos categorías: riesgo y no riesgo.

El objetivo del dataset fue proporcionar una base de prueba para evaluar la capacidad de distintos modelos de procesamiento de lenguaje natural para identificar posibles incumplimientos regulatorios a partir de información escrita.

Debido a que la actividad tiene fines académicos y de prototipado, el dataset fue construido utilizando ejemplos representativos de situaciones regulatorias comunes dentro de procesos de etiquetado y cumplimiento normativo. Esto permitió disponer de un conjunto de datos balanceado para realizar la comparación entre modelos preentrenados de Hugging Face.

## Estructura del dataset

El dataset está compuesto por dos campos principales:

| Campo | Tipo de dato   | Descripción                                                                                       |
|-------|----------------|---------------------------------------------------------------------------------------------------|
| text  | Texto (string) | Contiene el texto regulatorio o descripción que será analizada por el modelo.                     |
| label | Texto (string) | Indica la clasificación esperada para cada registro. Los valores posibles son "risk" y "no_risk". |

## Descripción de etiquetas

### risk

- Representa textos que contienen posibles incumplimientos regulatorios.
- Incluye ejemplos como ausencia de información obligatoria, ingredientes restringidos o incumplimiento de requisitos de etiquetado.

### no\_risk

- Representa textos que cumplen con los requisitos regulatorios establecidos.
- Incluye ejemplos donde la información se encuentra completa y alineada con la normativa aplicable.

## Distribución del dataset

El conjunto de datos utilizado contiene un total de 40 registros.

| Categoría | Cantidad |
|-----------|----------|
| risk      | 20       |
| no_risk   | 20       |
| Total     | 40       |

La distribución balanceada permite realizar una evaluación más objetiva de los modelos y evita sesgos derivados de una representación desproporcionada de alguna categoría.

## Scripts reproducibles

Con el fin de garantizar la reproducibilidad del experimento, se desarrolló un notebook en Google Colab utilizando Python y bibliotecas especializadas para inteligencia artificial.

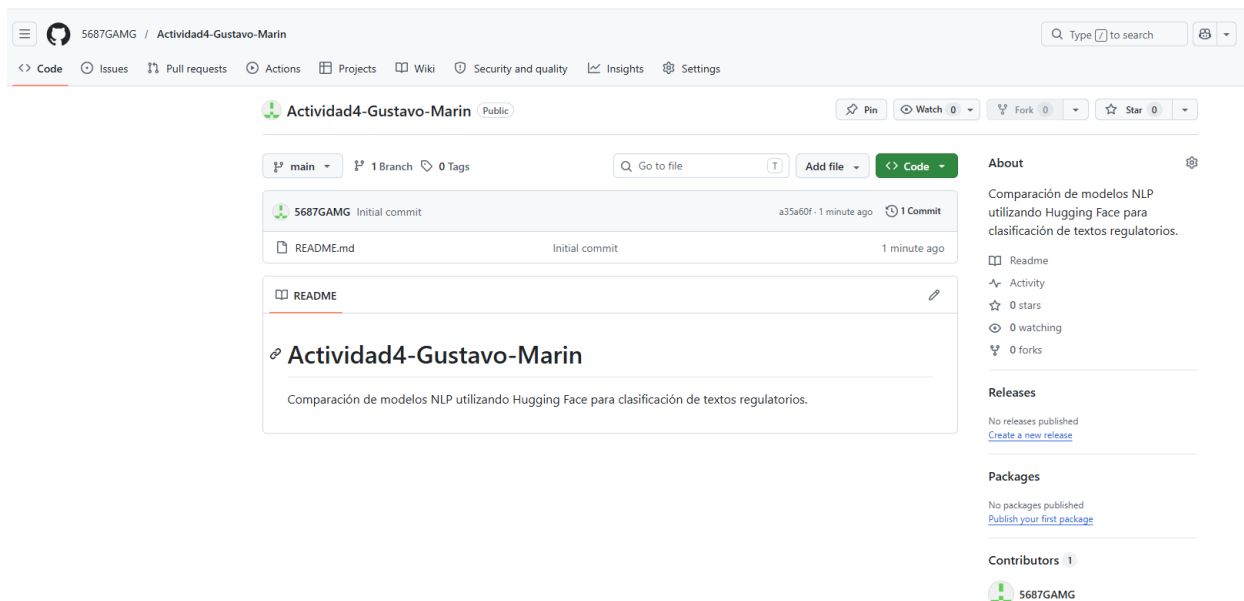
El procedimiento seguido fue el siguiente:

1. Instalación de dependencias necesarias (Transformers, Scikit-Learn, Pandas y Hugging Face Hub).

2. Configuración de GPU en Google Colab.
3. Autenticación con Hugging Face Hub.
4. Carga del dataset en formato CSV.
5. Selección de tres modelos preentrenados:
  - BART MNLI.
  - DeBERTa Zero-Shot.
  - DistilBERT MNLI.
6. Ejecución de inferencias utilizando pipelines de clasificación.
7. Cálculo de métricas de desempeño:
  - Accuracy.
  - Precision.
  - Recall.
  - F1-score.
8. Medición de latencia promedio.
9. Generación de tablas comparativas y visualizaciones.
10. Exportación de resultados para análisis posterior.

## 4.-Repositorio en GitHub.

Se crea repositorio llamado Actividad4-Gustavo-Marin



## Conclusiones :

El desarrollo de esta actividad me permitió comprender de manera práctica cómo utilizar modelos preentrenados de Hugging Face para resolver problemas relacionados con el análisis y clasificación de información textual. Más allá de la ejecución técnica de los modelos, el ejercicio ayudó a entender la importancia de evaluar distintos criterios antes de seleccionar una solución de inteligencia artificial para un entorno empresarial.

Los resultados obtenidos demostraron que cada modelo presenta fortalezas diferentes. Mientras algunos destacan por su velocidad de respuesta, otros ofrecen una mayor capacidad para identificar correctamente los casos de riesgo. Después de analizar las métricas obtenidas, se determinó que BART MNLI fue la alternativa que ofreció el mejor equilibrio entre precisión y capacidad de detección, convirtiéndose en la opción más adecuada para el caso de uso planteado.

Asimismo, la actividad permitió reforzar conceptos relacionados con la evaluación de modelos, la interpretación de métricas de desempeño, el uso de herramientas colaborativas como GitHub y la importancia de documentar adecuadamente los experimentos para garantizar su reproducibilidad. Considero que este ejercicio representa una aproximación muy cercana a situaciones reales que pueden presentarse en proyectos de inteligencia artificial dentro de las

organizaciones, donde no solamente importa la precisión del modelo, sino también aspectos como eficiencia, mantenimiento, riesgos y facilidad de implementación.

Finalmente, esta experiencia me permitió visualizar cómo tecnologías de procesamiento de lenguaje natural pueden integrarse en procesos de compliance y validación documental, generando oportunidades para mejorar la eficiencia operativa y fortalecer la toma de decisiones basada en datos dentro de un entorno empresarial.

### **Bibliografía:**

Hugging Face. (s.f.). *Hugging Face documentation*. Recuperado de <https://huggingface.co/docs>

Hugging Face. (s.f.). *Models*. Recuperado de <https://huggingface.co/models>

Google Colaboratory. (s.f.). *Welcome to Colaboratory*. Recuperado de <https://colab.research.google.com>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Howard, J., & Rush, A. M. (2020). *Transformers: State-of-the-art natural language processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>

Python Software Foundation. (s.f.). *Python documentation*. Recuperado de <https://docs.python.org/3/>